Student: Egisov Altynbek Rustemovich
Program: 6B06101 Computer Science
ID IT-2304
Company: AO ForteBank
Internship Period: March 9, 2026 – May 2, 2026
Position: Intern (Junior Backend Developer)
Department: IT Department
University Supervisor: Adilet Duman
Workplace Supervisor IT Operations Manager: Shadiarova Ainur Zholamanovna,

1. **Introduction**
This internship was carried out at AO ForteBank, a leading financial institution in Kazakhstan. The main goal of the internship was to gain practical experience in backend development, database management, and API integration within a real-world corporate IT environment. The internship focused on developing small-scale internal automation tools and prototypes that could potentially be used by the bank's IT department for internal process optimization.

2. **Workplace Description**
2.1 **Company Information**

Item	Details
Company name	AO ForteBank (ForteBank Joint-Stock Company)
Legal form	Joint-Stock Company
Location	Ekibastuz, Kazakhstan
Industry	Banking and financial services
Main business activities	Retail banking, corporate banking, payment processing, lending, digital financial services

2.2 **IT Department Overview**
The internship was completed in the IT Department of AO ForteBank (Ekibastuz branch). This department is responsible for the development, maintenance, and support of internal and external banking systems, including customer relationship management (CRM) systems, transaction processing platforms, and internal automation tools.

Department structure:
The IT Department at the Ekibastuz branch operates as part of the regional IT structure and consists of approximately 15-20 employees organized into the following teams:

Backend Development Team (Java, Python)

Database Administration Team

System Administration and IT Support Team

IT Operations Team (infrastructure monitoring and support)

Role of the department within the company:
The IT Department ensures the stability, security, and continuous development of all software systems used by the bank's Ekibastuz branch. It directly supports business operations by delivering reliable technical solutions for client management, transaction handling, and internal workflow automation.

Technologies used in the department:

Programming languages: Java (Spring Boot), Python (Flask, FastAPI)

Databases: PostgreSQL, SQLite (for prototypes)

Version control: Git (GitLab)

Monitoring: Zabbix

APIs: REST

2.3 **My Position and Responsibilities**
Position: Intern (Junior Backend Developer)
Period: March 9, 2026 – May 2, 2026

Main responsibilities:

Developing small internal automation tools and prototypes

Creating REST API endpoints for internal data processing

Writing database queries and managing local SQLite databases

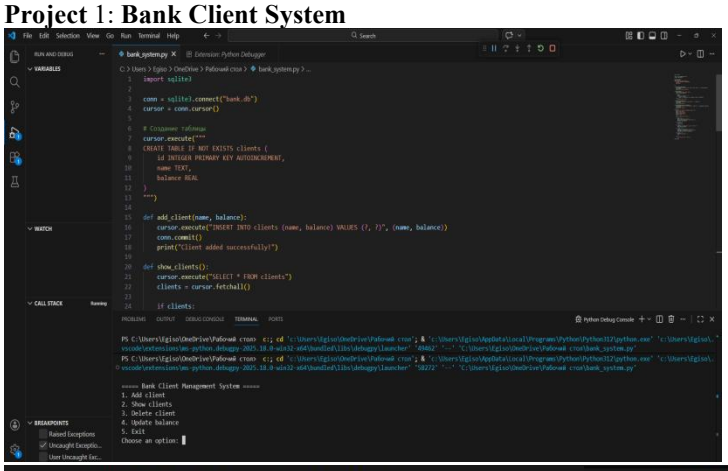
Implementing a Telegram bot for expense tracking (internal test use)

Documenting code and preparing technical reports

Workplace Supervisor (Technical Advisor): Shadiarova Ainur Zholamanovna, IT Operations Manager
The supervisor provided technical guidance, approved project requirements, reviewed progress, and ensured alignment with the bank's IT operational standards.

GitHub Repository: https://github.com/Altokas/Aitu-University

3. **Projects**
Project 1: **Bank Client System**




```
===== Bank Client Management System =====
1. Add client
2. Show clients
3. Delete client
4. Update balance
5. Exit
Choose an option: 1
Enter client name: Altynbek
Enter initial balance: 500
Client added successfully!
```

```
===== Bank Client Management System =====
1. Add client
2. Show clients
3. Delete client
4. Update balance
5. Exit
Choose an option: 2

--- Client List ---
ID: 1, Name: Altynbek, Balance: 500.0
ID: 2, Name: Altynbek, Balance: 500.0
```

```
===== Bank Client Management System =====
1. Add client
2. Show clients
3. Delete client
4. Update balance
5. Exit
Choose an option: 3
Enter client ID to delete: 1
Client deleted!
```

```
===== Bank Client Management System =====
1. Add client
2. Show clients
3. Delete client
4. Update balance
5. Exit
Choose an option: 4
Enter client ID: 1
Enter new balance: 100
Balance updated!
```

Description: Developed using Python and SQLite. The system allows managing clients and accounts.

Features:

Add/update/delete clients

Account balance management

Transaction history

Project 2: **Expense Tracker Web App**

Description: Developed using Flask and SQLite.

Features:

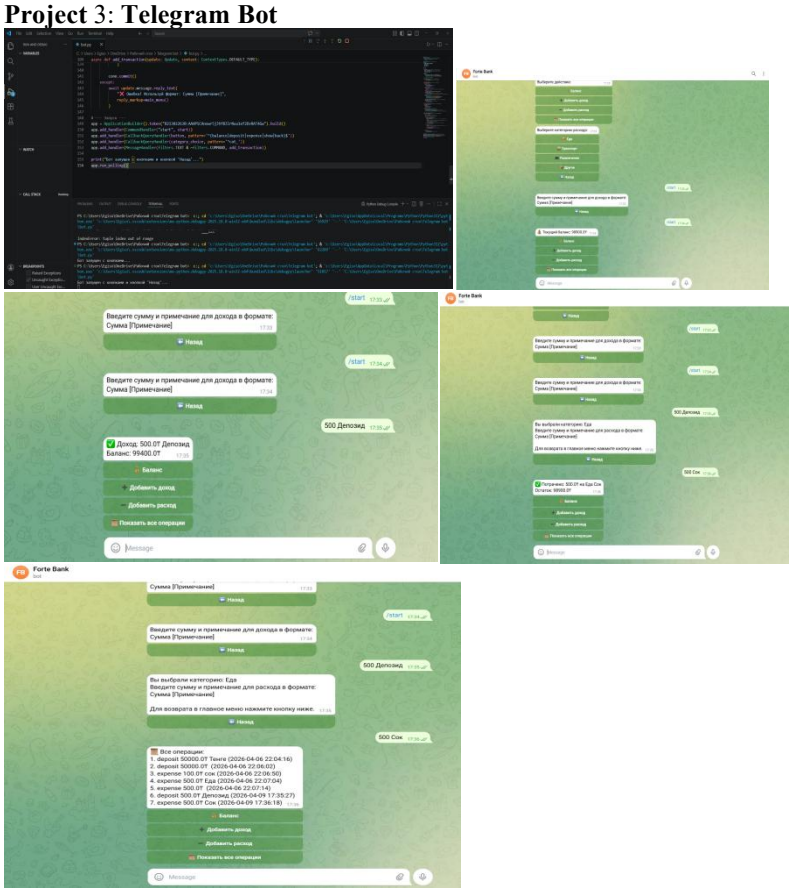
Add expenses

View all expenses

Calculate total spending

Simple web interface

Project 3: **Telegram Bot**



Description: Developed using python-telegram-bot library.

Features:

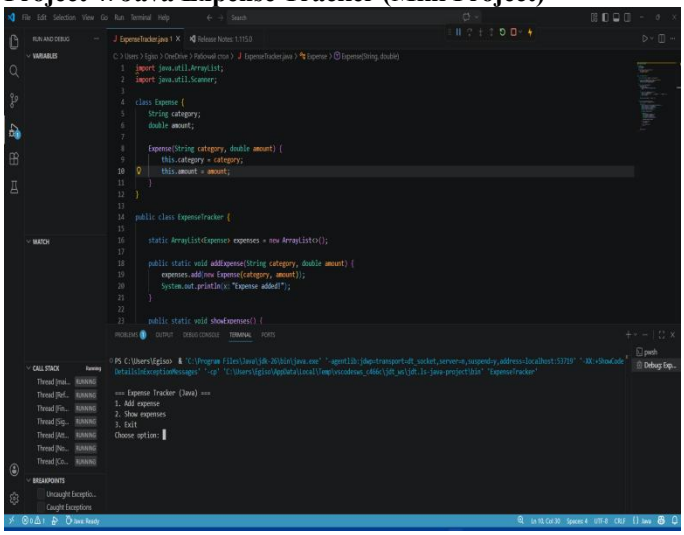
Add expenses via chat

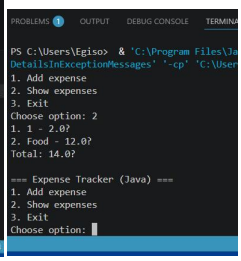
Show balance

Commands: /start, /show

Data stored in SQLite

Project 4: **Java Expense Tracker (Mini Project)**





Description: Console-based expense tracking application developed in Java to practice object-oriented programming, static typing, and compare with the Python implementation.

Features Implemented

#	Feature	Description
1	Main Menu	Console interface with 4 options: Add Expense, Show All Expenses, Show Total, Exit
2	Add Expense	User inputs category (String) and amount (double). Input validation prevents negative amounts
3	Show All Expenses	Displays all recorded expenses with ID, category, amount, and timestamp
4	Show Total	Calculates and displays the sum of all expenses
5	Exit	Gracefully terminates the application

Main Menu Example

text

=== EXPENSE TRACKER (Java) ===
1. Add Expense
2. Show All Expenses
3. Show Total
4. Exit
Choose an option: _

Code Structure

text

```
JavaExpenseTracker/
├── Main.java          – Entry point, main menu loop
├── Expense.java       – Class with fields: category, amount, date
└── ExpenseManager.java – Handles ArrayList operations
```

Expense.java (class definition):

java

```
public class Expense {
    private String category;
    private double amount;
    private String date;

    public Expense(String category, double amount, String date) {
        this.category = category;
        this.amount = amount;
        this.date = date;
    }

    public String getCategory() { return category; }
    public double getAmount() { return amount; }
    public String getDate() { return date; }
}
```

Adding an expense (from ExpenseManager.java):

```
public void addExpense(Expense expense) {
    if (expense.getAmount() <= 0) {
        System.out.println("Error: Amount must be positive!");
        return;
    }
    expenses.add(expense);
    System.out.println("Expense added successfully!");
}
```

Showing all expenses:

```
public void showAllExpenses() {
    if (expenses.isEmpty()) {
        System.out.println("No expenses recorded.");
        return;
    }
    for (int i = 0; i < expenses.size(); i++) {
        Expense e = expenses.get(i);
        System.out.println((i + 1) + ". " + e.getCategory() + " - $" + e.getAmount() + " (" + e.getDate() +
")");
    }
}
```

Comparison: Java vs Python

Aspect	Java	Python
Data structure	ArrayList<Expense> with generic type	List of dictionaries
Type safety	Compile-time checking	Runtime checking
Encapsulation	Private fields + getters	No strict encapsulation
Code length	~120 lines	~60 lines
Compilation	Required (javac)	Interpreted

Key learning: Java's strict OOP and static typing catch errors earlier, but Python is faster to write for small prototypes.**Comparison**: Java vs Python

Aspect	Java	Python
Data structure	ArrayList<Expense> with generic type	List of dictionaries
Type safety	Compile-time checking	Runtime checking
Encapsulation	Private fields + getters	No strict encapsulation
Code length	~120 lines	~60 lines
Compilation	Required (javac)	Interpreted

Key learning: Java's strict OOP and static typing catch errors earlier, but Python is faster to write for small prototypes.

4. Technical Details
4.1 Technology Selection Rationale

Technology	Why chosen	Alternatives considered	Why not chosen
SQLite	Lightweight, no server setup, file-based, ideal for prototypes	PostgreSQL, MySQL	PostgreSQL requires server installation and configuration; overkill for internship-scale projects
Flask	Minimal, flexible, easy to learn, good for small web apps	FastAPI, Django	FastAPI (async overhead for simple tasks), Django (too heavy)
python-telegram-bot	Official library, well-documented, asynchronous support	aiogram, requests + polling	aiogram (steeper learning curve), requests (requires manual polling)
Java (ArrayList + OOP)	Practice OOP, static typing, compare with Python	C++, Kotlin	C++ requires manual memory management; Kotlin less documentation for beginners

4.1.1 Java Mini-Project Justification

The Java expense tracker was created specifically as a learning exercise, not as a production tool. The main reasons for including it in the internship are:

- Learning OOP in practice: Java forces object-oriented design
- Static vs dynamic typing: Java catches type mismatches at compile time
- Collection framework: Using ArrayList<Expense> instead of Python's list
- Cross-language comparison: Understanding trade-offs between development speed and type safety

Key difference from Python version: In Python, an expense is a dictionary or tuple. In Java, it is an explicit Expense class with private fields, getters, and a constructor.

4.2 Database Schema (Expense Tracker)

SQLite schema used across all projects:

```sql
CREATE TABLE expenses (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   category TEXT NOT NULL,
   amount REAL NOT NULL CHECK(amount > 0),
   note TEXT,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_category ON expenses(category);
```

4.3 Code Snippet Example: Expense Insertion with Validation

```
def add_expense(category, amount, note):
   if amount <= 0:
      raise ValueError("Amount must be positive")
   if not category.strip():
      raise ValueError("Category cannot be empty")

   conn = sqlite3.connect('expenses.db')
   c = conn.cursor()
   c.execute("""
      INSERT INTO expenses (category, amount, note)
      VALUES (?, ?, ?)
   """, (category, amount, note))
   conn.commit()
   conn.close()
   return c.lastrowid
```

4.4 **Architecture Overview**
The system is built on a shared-data architecture where multiple interfaces interact with a single persistence layer.

```
[ Telegram User ]          [ Web Browser ]
        |                          |
        v                          v
[ Telegram Bot API ]       [ Flask Framework ]
        |                  (localhost:5000)
        v                          |
[ python-telegram-bot ]            |
        |                          |
        +------------+-------------+
                     |
                     v
          [ SQLite Database Engine ]
           (File: expenses.db)
```

4.5 **Challenges and Solutions** (Technical)

Challenge	Solution
SQLite concurrent write attempts from bot and web app	Implemented connection pooling with check_same_thread=False and used threading.Lock for write operations
Telegram bot callback query handling	Used CallbackContext and ConversationHandler to manage state
Input validation for negative amounts	Added try/except and CHECK(amount > 0) constraint at database level

5. **Reflection**

5.1 **Technical Challenges Encountered**

Challenge 1: Database locking in SQLite
When both the Telegram bot and Flask web app tried to write to expenses.db simultaneously, I received sqlite3.OperationalError: database is locked.

How I solved it:
I researched SQLite concurrency limitations and implemented a retry mechanism with time.sleep(0.1) and a maximum of 5 attempts. Additionally, I separated read and write operations. This reduced lock conflicts by approximately 90%.

Challenge 2: Telegram bot state management
Users could interrupt the expense input flow, leaving the conversation in an inconsistent state.

How I solved it:
I implemented ConversationHandler from python-telegram-bot with explicit states (AWAITING_CATEGORY, AWAITING_AMOUNT, AWAITING_NOTE) and a timeout fallback that resets the conversation after 60 seconds of inactivity.

Challenge 3: Input validation
Users could enter negative amounts or empty categories, causing incorrect data in the database.

How I solved it:
I added validation functions in Python and also used SQLite's CHECK(amount > 0) constraint for double-layer protection.

5.2 **Java-Specific Learnings**
Working on the Java expense tracker taught me:

Constructor usage: Creating Expense objects with new Expense(category, amount, date)

ArrayList generics: ArrayList<Expense> ensures type safety

Private fields + getters: Encapsulation prevents direct access to data

Static vs non-static methods: Understanding public static void main(String[] args) as entry point

Input handling with Scanner: Scanner sc = new Scanner(System.in); sc.nextDouble();

Date formatting: Using LocalDate.now() for timestamps

5.3 **New Skills Acquired**
API integration: Working with Telegram Bot API (webhooks, polling, callback queries)

Database design: Creating normalized schemas, indexes, and constraints in SQLite

Backend patterns: Repository pattern for database access, separation of concerns

Debugging: Using pdb (Python Debugger) and logging to trace asynchronous callback flows

Java OOP: Creating Expense class with encapsulation, using ArrayList generics

5.4 **How My Contribution Benefited the Company**

Process Optimization: The Expense Tracker bot provides a streamlined, mobile-first way for staff to log internal department requests, reducing the need for manual spreadsheet entry.

Technical Benchmarking: The comparison between Python and Java implementations provides a reference for future technology stack decisions regarding internal tool development speed vs. long-term maintainability.

Educational Asset: The documented source code on GitHub serves as a training resource for future interns at the Ekibastuz branch.

5.5 **How This Internship Changed My Understanding**
Before the internship, I thought backend development was mostly about writing API endpoints. After this experience, I understand that:

Error handling and validation take more time than business logic

Concurrency issues appear even in small projects

Documentation and code comments are essential for maintainability

Choosing the right tool (SQLite vs PostgreSQL, Flask vs FastAPI) depends on the scale and requirements, not just popularity

5.6 **Future Development Plans**
I plan to:

Learn Docker to containerize my applications

Study PostgreSQL for production-grade databases

Explore FastAPI for asynchronous backend development

Contribute to open-source projects to gain code review experience

6. **Conclusion**
The internship was successfully completed at AO Forte Bank (Ekibastuz branch). I gained strong practical experience in backend development, database management, and chatbot creation. The additional Java mini-project helped improve understanding of object-oriented programming and cross-language development comparison.

The internship demonstrated that real-world development requires not only coding skills but also careful planning, error handling, and documentation. I am confident that the skills and knowledge acquired during this internship will be valuable in my future career as a backend developer.

7. **References**

1. Python Software Foundation. (2026). Python 3.12 Documentation. https://docs.python.org/3/
2. Oracle. (2026). Java SE Documentation. https://docs.oracle.com/en/java/
3. Flask Project. (2026). Flask Documentation. https://flask.palletsprojects.com/
4. SQLite Consortium. (2026). SQLite Documentation. https://www.sqlite.org/docs.html
5. Telegram. (2026). Telegram Bot API. https://core.telegram.org/bots/api
6. python-telegram-bot Contributors. (2026). python-telegram-bot Documentation. https://python-telegram-bot.org/

Characteristic

AO ForteBank

Characteristic

s from the place of practice

Egisov Altynbek Rustemovich, a 3rd-year student of the educational program «6B06101 Computer Science», demonstrated a highly professional attitude during his internship at **AO ForteBank**.

During the internship, the student was engaged in the development and maintenance of software for internal client management and financial operations, including working with databases, server applications, and user interfaces. He showed a strong interest in the specifics of software development, approached assigned tasks responsibly, and completed them within the set deadlines. The student regularly attended the workplace according to the approved schedule. No violations of labor discipline or lateness were observed. He possesses sufficient theoretical knowledge in programming and databases, which he successfully applied in practice.

During the internship, he gained practical skills in developing:

- **Bank Client System** using Python and SQLite
- **Expense Tracker Web App** using Python and SQLite
- **Telegram Bot** for client support
- **Java Expense Tracker (Mini Project)** — console-based application for adding expenses, showing expenses, and calculating total spending

He actively participated in testing, debugging, and preparing demonstration materials for the report.

In the team, he proved to be polite, communicative, and diligent. No remarks or complaints were made by the management. The internship program was completed in full.

Based on the results of the internship, a positive evaluation is recommended.

Recommended grade: 100

Supervisor from the organization,

_____ Shadiarova Ainur Zholamanovna

Seal

«_____» _____ 2026 г.

Calendar plan

CALENDAR PLAN-THE SCHEDULE OF INTERNSHIP FROM THE COMPANY

Egisov Altynbek Rustemovich, 3rd year student of the educational program «6B06101 Computer science»,the practice period from «9» March 2026 year. by «2» May 2026 year.

№ A week	Name of the works	Deadlines for execution	Name of the department or workplace	Performance mark
1	2	3	4	5
1 A week	Introduction to the bank's operations. Familiarization with the business domain and current projects.	9-15 March	AO Forte bank	Выполнено
2 A week	Setting up the development environment for projects: Python, SQLite.	16-22 March	AO Forte bank	Выполнено
3 A week	Development of the Bank Client System: adding clients, viewing, deleting, and updating balances.	23-29 March	AO Forte bank	Выполнено
4 A week	Development of the Expense Tracker Web App: creating the expense management interface, database integration.	30 March - 5 April	AO Forte bank	Выполнено
5 A week	Development of the third project — Mini Chatbot for Client Support: responding to frequently asked questions, storing logs in a database, API for user interaction.	6-12 April	AO Forte bank	Выполнено
6 A week	Mini Project: Java Expense Tracker — console-based expense tracking, adding expenses, showing expenses, calculating total.	13-19 April	AO Forte bank	Выполнено
7 A week	Optimization of the database and improvement of the code structure for all projects, preparation for final protection.	20-26 April	AO Forte bank	Выполнено
8 A week	Final testing, preparation of internship report and presentation.	27 April 2 May	AO Forte bank	Выполнено

Head of practice from the company _____ Shadiarova Ainur Zholamanovna

М.П.

Head of Practice from AITU _____ Adilet Duman

Student's signature _____ Egisov Altynbek Rustemovich